

Using Xalan to achieve application interoperability

Chris McManaman
RCG Information Technology



Outline

- Integration Solution Case Study
- XML as an integration tool
- Xalan as an integration tool
- Putting the peddle to the metal with XSLT
- XSLT Facilitation on an integration project

Integration Case Study

- Client Needs
  - Duplicate data entry
  - Frequent batch processing failures
  - Maintenance for early morning batch processing
  - Inconsistent data throughout applications



Integration Case Study

- Create a real time global company.



Why use XML to do this?

- Multiple records in one xml string.
- Easier to query and manipulate.
- Context allows users and developers to identify problems within data.



XML and Integration

- XML allows you to send multiple records within one transaction
- Zipping the XML string enables large transactions without hindering performance



Why XSLT to do this?

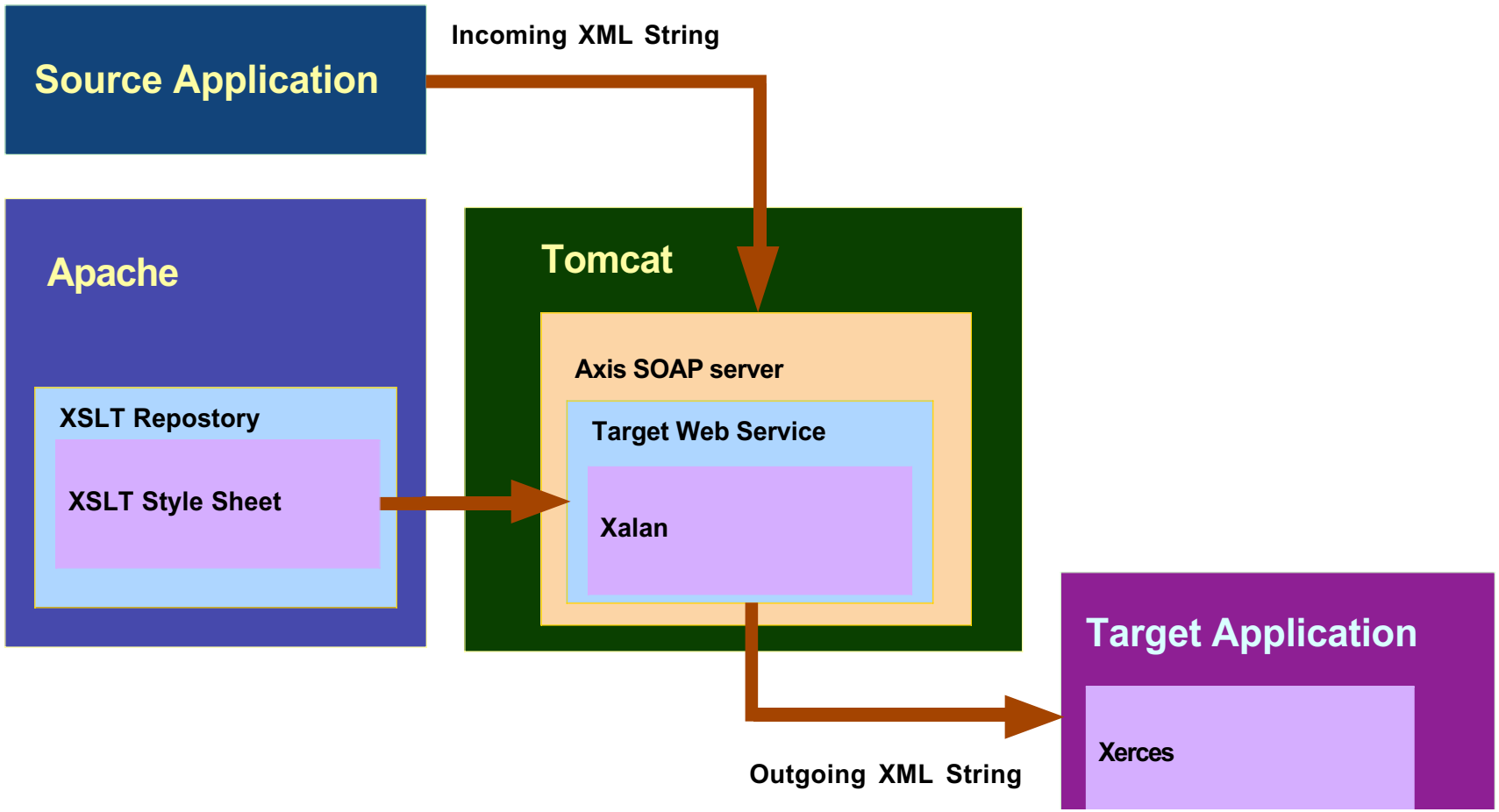
- Avoid the “blame the middleware” game
- XSLT gives control and responsibility to the source and target applications
- XSLT allows one xml to be used and customized by different applications
- Administration of XSLT style sheets is easier to maintain than coding the transformation



XSLT Repository

- We created an XSLT repository that allowed developers to update and publish XSLT style sheets.
- The style sheets could updated instantly.

Service Oriented Architecture





What is XML

- XML: e**X**tensible **M**arkup **L**anguage

XML is a **framework** for defining markup languages

- there is **no fixed collection of markup tags** - we may define our own tags, tailored for our kind of information
- each XML language is targeted at its own application domain, but the languages will share many features
- there is a common set of **generic tools** for processing documents

Purposes of XML

- separate **syntax** from **semantics** to provide a common framework for structuring information (browser rendering semantics is completely defined by style sheets);
- allow **tailor-made markup** for any imaginable application domain
- support **internationalization** (Unicode) and **platform independence**
- be the future of structured information, including **databases**



The Many Parts of XML

- **XSLT** will be used to transform the XML document into XHTML (or HTML), including automatic construction of index, references, etc.
- **XLink**, **XPointer**, and **XPath** could be used to create cross-references

XSLT - XSL Transformations

- **XSL** (e**X**tensible **S**tylesheet **L**anguage) consists of two parts:
  - *XSL Transformations* (**XSLT**)
  - *XSL Formatting Objects* (**XSL-FO**)



XSLT

- A transformation expressed in XSLT describes rules for transforming a source tree into a result tree.
- The transformation is achieved by associating patterns with templates.
- A pattern is matched against elements in the source tree. A template is instantiated to create part of the result tree.



What is Xalan?

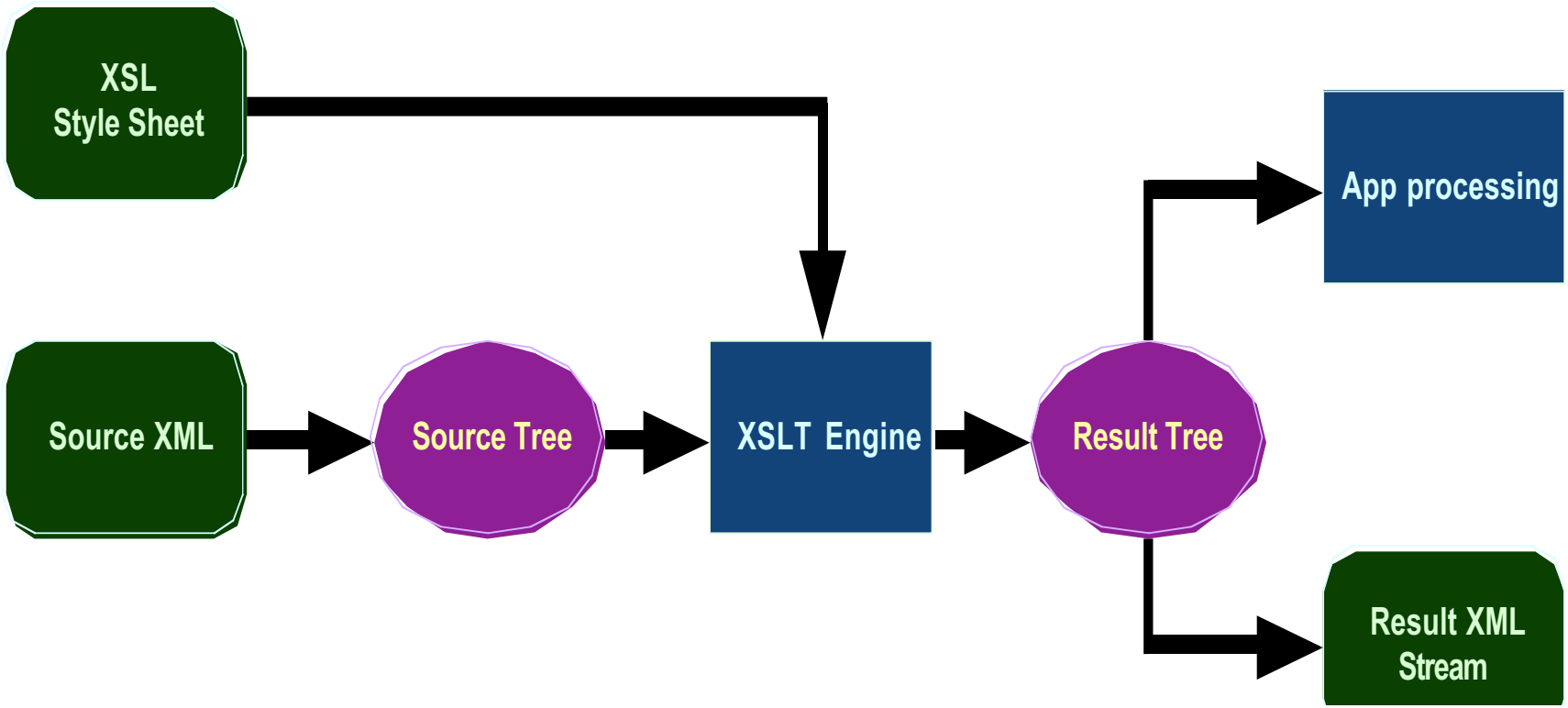
- Xalan-Java is an XSLT processor for transforming XML documents into other XML document types.
- It includes the XSL Transformation vocabulary and XPath, a language for addressing parts of XML documents.
- It can be used from the command line, or as a module in another program.



Xalan Features

- Interpretive processor
- Works with any XML parser (Xerces)
- Can process Stream, SAX or DOM input, and output to a Stream, SAX or DOM.
- Transformations may be chained
- May be run from the command line
- Supports the creation of Java extensions

Architecture





Xalan-Java API

- **Transformations API for XML (TrAX)**
- **Java API for XML Parsing**
- **Xalan Core**
- **XPath**
- **Document Table Model (DTM)**
- **Utilities**
- **Xalan Other**
- **Xalan Extensions**
- **Serializers**
- **SAX 2**
- **DOM 2**
- **XSLTC Core**
- **Other Packages**

Basic steps

```
javax.xml.transform.TransformerFactory tFactory =
    javax.xml.transform.TransformerFactory.newInstance()

javax.xml.transform.Transformer transformer = tFactory.newTransformer (new
    javax.xml.transform.stream.StreamSource(“stylesheet.xsl");

transformer.transform (new
    javax.xml.transform.stream.StreamSource(“incoming.xml"), new
    javax.xml.transform.stream.StreamResult( new
    java.io.FileOutputStream(“outgoing.xml")));
```



Xalan Debugging Options

- -XSLTC
  - Use xsltc for transformation
- -IN
  - input xml
- -XSL
  - Location of the xslt style sheet
- -OUT
  - Name of the output file

Xalan Debugging Options

- -TT
  - Trace the templates as they are being called.
  - file:///C:/xslt_pres/in.xsl Line #3, Column #28: template match='/‘
- -TG
  - Trace each generation event.
    - STARTDOCUMENT
    - STARTELEMENT: well_number
    - CHARACTERS: 90003039
    - ENDELEMENT: well_number
    - ENDDOCUMENT
- -TS
  - Trace each selection event.
  - file:///C:/xslt_pres/in.xsl Line #8, Column #46: if, test='starts-with(well_no,'9')': true

Xalan Debugging Options

- -TTC
  - Trace the template children as they are being processed.
    - template match='/'
    - apply-templates
    - template match='root/daily'
    - element
    - if
    - value-of



Xalan Core

- org.apache.xalan.processor
  - The module that processes the stylesheet, and provides the main entry point into Xalan.



Xalan Core

- org.apache.xalan.templates
  - The module that defines the stylesheet structures, including the Stylesheet object, template element instructions, and Attribute Value Templates.



Xalan Core

- org.apache.xalan.transformer
  - The module that applies the source tree to the Templates, and produces a result tree.



Xalan Core

- org.apache.xpath
  - The module that processes both XPath expressions, and XSLT Match patterns.



Processing model

- An XSLT stylesheet consists of a number of **template rules**:
  - **template rule = pattern + template**

Processing model

- An XML document will be processed through XSLT by :
  - The XML will be processed starting at the root node and working its way down the document to construct a tree.
  - A single **node** is processed by:
    - **finding** the template rule with the best matching pattern
    - **instantiating** its template (creates result fragment + continues processing recursively)

Location & Step Patterns

- ‘/’ defines:
  - a parent child relationship
  - A separator between step patterns
- Root/daily
  - We need to find header elements that are children of the root element.
- Root//volume
  - Find <volume> elements that are nested to any level.



Patterns

- We traverse the **source tree** to find **node-sets** that ***match a pattern*** determine by an **XPath expression**



Patterns

- A given node **matches** a given pattern if:
  - the node is member of the result of **evaluating** the pattern with respect to **some** context.
- Operationally (and intuitively), a pattern matching is probably best evaluated backwards (from right to left).



Expressions

- Types:
  - **node-set**
  - **boolean**
  - **number**
  - **string**

Expressions

- An **expression** can be:
  - a constant, e.g. **"..."**
  - a variable: **$*variable***
  - a function call: ***function*** **(** ***arguments*** **)**
  - a boolean expression: **or**, **and**, **=**, **!=**, **<**, **>**, **<=**, **>=** (standard precedence, all left associative)
  - a numerical expression: **+**, **-**, *, **div**, **mod**
  - a node-set expression (using **location paths**!): **|** (set union)

Functions

- Manipulating Numbers
  - Floor()
  - Ceiling()
  - Round()
  - Format-number(number,format)
- Manipulating Strings
  - Substring-before(string,location)
  - Substring-after(string,location)
  - Substring(string, location)
  - Concat(value,value,…)
  - String-length(string)



XSLT templates

- A template is instantiated for a particular source element to create part of the result tree.
- A template can contain elements that specify literal result element structure.
- A template can also contain elements from the XSLT namespace that are instructions for creating result tree fragments.



XSLT Templates

- When a template is instantiated, each instruction is executed and replaced by the result tree fragment that it creates.
- Elements are only processed when they have been selected by the execution of an instruction.
- The result tree is constructed by finding the template rule for the root node and instantiating its template.



Templates

- There are many different kinds of template constructs:
  - literal result fragments
  - recursive processing
  - computed result fragments
  - conditional processing
  - sorting
  - numbering
  - variables and parameters
  - keys

Literal result fragments

- A literal result fragment is:
  - a text constant (character data)
  - an element not belonging to the XSL namespace
  - **<xsl:text ...> ... </...>**
  - **<xsl:comment> ... </...>**

Recursive processing

- Recursive processing instructions:
  - **<xsl:apply-templates select="*node-set expression*" .../>**
  - **<xsl:call-template name="..."/>**
  - **<xsl:for-each select="*node-set expression*"> *template* </...>**
  - **<xsl:copy> *template* </...>**
  - **<xsl:copy-of select="..."/>**
- The **select** attribute contains an **XPath expression**, which is evaluated in the current context.



Recursive processing

- **Processing modes: mode="..."** on **xsl:template** and **xsl:apply-templates** allows an element to be processed multiple times in different ways.

Creating the XML stream values

- **The following XPath expressions create the resulting xml stream**
  - **<xsl:element name="..." namespace="..."> ... </...>**
  - **<xsl:attribute name="..." namespace="..."> ... </...>**
  - **<xsl:value-of select="..."/>**
    <xsl:processing-instruction name="..."> ... </...>

Conditional processing

- **Processing can be conditional:**
  - **<xsl:if test="*expression*"> ... </...>**
  - **<xsl:choose>**
    <xsl:when test="*expression*"> ... </...>
    ...
    <xsl:otherwise> ... </...>
    </...>

Sorting

- Sort the order of elements by :
  - **<xsl:sort select="*expression*" .../>**;
  - **Sorting attributes**:
    - order="ascending/descending"
    - lang="..."
    - data-type="text/number"
    - case-order="upper-first/lower-first"
- **Sorting can only be used within:**
  - **xsl:apply-templates**
  - **xsl:for-each**

Variables and parameters

- for reusing results of computations and parameterizing templates and whole stylesheets
  - static scope rules
  - can hold any XPath value (string, number, boolean, node-set) + **result-tree fragment**
  - purely declarative: variables cannot be updated
  - can be global or local to a template rule
- Declaration:
  - **<xsl:variable name="..." select="*expression*"/>** variable declaration, value given by XPath expression
  - **<xsl:variable name="..."> *template* </..>** variable declaration, template is instantiated as result tree fragment to give value



Variables and parameters

- To access and pass variables within a stylesheet use:
  - **$*name***
  - **xsl:with-param**



Accessing Java from XSLT

- xmlns:javafunc="java.lang“
- exclude-result-prefixes="javafunc“

```
<xsl:value-of select="javafunc:System.currentTimeMillis()"/>
```



XSLT Facilitation

- Start small
- Work with the developers not for them
- Test their XSLT’s = Don’t let them fail

This presenation has ended.
Click anywhere to return.

_

